

Computer Program

- A sequential set of instructions, known as codes, written or “programmed” in a computer language to let the computer perform a specific computation task.
- Examples include system software, web browsers, utility software, multimedia software, and spreadsheet software.
- The set of instructions in a computer program must be performed sequentially unless directed otherwise.
- The instructions in the set will express a unit of work that a computer language can support.

Computer Language / Programming Language

- A set of grammatical rules that commands a computer or a device to behave in a specific way.

Programming

- It is the art and science of instructing computers to perform tasks using a specific programming language.
- It creates codes that tell a computer how to solve problems and accomplish various operations.
- By writing code in programming languages, programmers can develop computer programs or software applications, design websites, analyze data, and automate processes, among other operations.

Evolution and History of Programming

- **Machine Code:** Consisted of binary instructions (0 and 1) directly communicating with the computer’s hardware.
- **High-level Programming Languages:** Developed to simplify the coding process. These languages allowed programmers to write code using more human-readable syntax.

Programming Paradigms

- The term “paradigm” is synonymous with “pattern.” These refer to the different approaches to structuring and organizing code. These provide a conceptual framework for solving problems and designing software.
- **Procedural Programming:** Or imperative programming, focuses on organizing code into reusable procedures or functions, emphasizing the sequence of steps to execute a program. Examples include BASIC, C and C++, Pascal, and Java.
- **Object-oriented programming (OOP):** Revolves around the concepts of objects that encapsulate data and behavior that promotes modularity, code reusability, and scalability. Examples include Python, VB.NET, and C#.
- **Functional Programming:** Treats computation as evaluating mathematical functions, emphasizing consistency, and avoiding side effects

Programming Languages

- Numerous programming languages are available, each with its unique syntax, features, areas of specialization, and community support, making it suitable for specific projects.
- This can be classified into two (2) Low-level and high-level languages.

Classification of Programming Languages

- **Low-level Language:** A programming language closer to machine code and hardware in terms of syntax. It provides direct control over the computer's hardware and resources, allowing programmers to write code at a more detailed level. It is commonly used for tasks requiring precise control and efficient execution.
 - **Assembly Language:** Uses specific instructions to control a computer's hardware.
 - **Machine Language:** All instructions are written as binary numbers (1 and 0).
- **High-level Language:** A programming language designed to be easy for humans to read, write, and understand, allowing programmers to write computer programs and interact with a computer system without needing specific knowledge of the processor or hardware that the program will run on. Examples include C++, Pascal, PHP, Python, and Java.

Other Programming Terminologies

- **Syntax:** It is a set of rules defining the various combinations and arrangements of symbols or characters to create a valid statement in a language.
- **Command:** It is the unique instruction given to a computer application to perform a task of a function such as "print" to display text on the screen.
- **Integrated Development Environment (IDE):** It is a software application for formatting the code, checking the syntax, and running and testing the code. IDEs can work with multiple programming languages, while some are specific to only one language.
- **Library:** It is a collection of useful resources, such as objects and functions, that can be used individually and must be configured to work together. It is pre-built or installed in an IDE or Integrated Development Environment.
- **Interpreter:** It is a program that directly executes instructions in a high-level language without converting them into a machine language.
- **Assembler:** It is a program that converts instructions written in low-level assembly code into relocatable machine language.
- **Compiler:** It is a program that converts high-level languages into machine-readable code that a computer can execute.

Algorithm

- In programming, an algorithm is a set of steps that generates a finite sequence of simple computational operations leading to the solution of a given problem.

- It must be expressed in a natural language that anyone can follow, such as directions that can be written in English.

Algorithm Example: Finding Volume of a Rectangle

- Formula: $\text{volume} = \text{length} \times \text{width} \times \text{height}$
1. Get the length of the rectangle
 2. Get the width of the rectangle
 3. Get the height of the rectangle
 4. Find the volume using the formula: $\text{volume} = \text{length} \times \text{width} \times \text{heights}$
 5. Display the computed volume

Problem-Solving Process Steps in Programming

1. **Problem Analysis:** Evaluating and outlining the problem and its solution requirements
 2. **Algorithm Design:** Designing an algorithm to solve problems.
 3. **Coding:** Implementing the algorithm in a programming language.
 4. **Execution:** Verifying whether the algorithm works or not.
- **Note:** The two (2) commonly used tools in representing algorithms are through a pseudocode or a flowchart.

Pseudocode

- It is a technique used to describe distinct steps of an algorithm that is much easier to understand for anyone with basic programming knowledge.

Rules for Writing Pseudocode

- **Symbols used for common operations:**
 - **Arithmetic operations:** (+, -, *, /)
 - **Assignment:** (=)
 - **Comparison:** (=, ≠, <, >, ≤, ≥)
 - **Logical:** (and, or)
- **Keywords:** Can be used as a command, such as PRINT, WRITE, READ, SET, and GO TO.
- **Indentation:** Used to indicate branches and loops of instructions.

Pseudocode Example: Finding Volume of a Rectangle

- READ length
- READ width
- READ height
- SET volume to 0
- COMPUTE volume as $\text{length} * \text{width} * \text{height}$
- PRINT volume

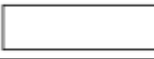
Flowchart

- It is a diagrammatic representation of the steps of an algorithm.

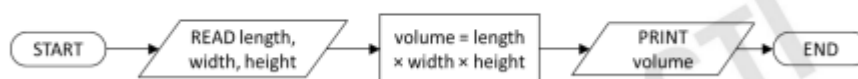
- It is a pictorial representation that can be used as a substitute for an algorithm, as a textual description of an algorithm may not be understood easily.
- Boxes of different shapes are used to represent various types of operations. Lines then connect these boxes with arrows representing the direction or flow to which one should proceed to know the next step.

Categories of Flowcharts

- **Program flowcharts:** Illustrate the logical steps in a software program or programming task to understand a process, workflow, or algorithm. They contain the steps of solving a problem unit for a specific result.
- **System flowcharts:** Show how parts of a system work together by displaying the data flow and how decisions can affect the events surrounding it.

Symbol	Name	Description
	Terminal	Shows the start and end of a set of computer-related processes
	Input/Output	Shows any input/output operation
	Computer Processing	Shows any processing performed by a computer system
	Predefined Processing	Indicates any process not specially defined in the flowchart
	Comment	Used for writing any explanatory statement required to clarify something
	Flow line	Used for connecting the symbols
	Document Input/Output	Used when input comes from a document, and output goes to a document
	Decision	Shows any point in the process wherein a decision must be made to determine further action
	On-page Connector	Connects parts of a flowchart continued on the same page
	Off-page Connector	Connects parts of a flowchart continued to separate pages

The flowchart to find and display the volume of the rectangle is:



Five (5) Rules for Creating Program Flowcharts

- **Rule 1:** Only the standard symbols should be used in program flowcharts.
- **Rule 2:** The program logic should only show the flow from top to bottom and/or left to right.

- **Rule 3 (Single Rule):** Each symbol should contain only one entry point and one exit point, except the decision symbol.
- **Rule 4:** The operations shown within a symbol should be expressed independently of any programming language.
- **Rule 5:** All decision branches should be well-labeled.